

Succinct Rabung certificates and recurrence-closed lower bounds for van der Waerden numbers

Brice Pouly*

Abstract

We present three primitive Rabung certificates at three colours, giving nine direct lower bounds for van der Waerden numbers $W(3, k)$ with $17 \leq k \leq 25$. The five bounds for $17 \leq k \leq 21$ improve the earlier comparators identified in a source audit updated on 26 September 2026, including the relevant specializations of a stronger published statement whose proof status remains unresolved in that audit. In particular, we obtain $W(3, 17) > 31\,385\,622\,833$, improving the identified predecessor by a factor of 2.02. The GPU search supplies candidate triples (p, r, k) ; a stand-alone CPU verifier reconstructs the finite hypotheses of Rabung’s theorem. We prove the equivalence of the implemented boundary test with the published criterion and provide a provenance-bearing finite closure under known recurrences. Four earlier two-colour certificates from Monroe’s distributed project are re-verified and credited to that project. An empirical comparison of search intensities is reported separately and is not used to establish the lower bounds.

Keywords. Van der Waerden numbers; power-residue colourings; Rabung certificates; computational Ramsey theory.

*Prepared with artificial-intelligence assistance; see the declaration at the end.

Contents

1	Introduction	2
2	Main results and exact scope	4
3	The Rabung criterion and certificates	8
4	GPU discovery campaign	13
5	Conclusion	15
6	Data, code, and reproducibility	15
A	Repository-recorded holdout protocol and evidence boundary	17
B	Holdout check for run-valid $(3, 17)$ primes	18
C	Retrospective diagnostics for the holdout comparison	20
D	Certificate evidence identities	20
E	Source catalogue and comparison scope	22
F	Campaign coverage audit	23
	Acknowledgements	25
	Declaration on artificial-intelligence assistance	25
	References	25

1 Introduction

A lower bound for a van der Waerden number is witnessed by a colouring that avoids long monochromatic arithmetic progressions. At the sizes considered here, such a colouring contains tens of billions of positions. Its description, however, need not be comparably large. In Rabung’s power-residue construction, a triple of integers (p, r, k) determines the finite conditions from which the colouring and its lower bound follow. The computational problem is to find a useful triple; the mathematical problem is to verify those conditions and to place the bound among the consequences of earlier constructions.

For integers $r \geq 2$ and $k \geq 3$, let $W(r, k)$ be the least N such that every r -colouring of $\{1, \dots, N\}$ contains a monochromatic arithmetic progression of length k . We put colours first. Monroe [18] uses the reverse convention $W(k, r)$; Fox and Hunter [4] write $w(k; r)$.

1.1 Numerical results and their scope

We obtain three primitive three-colour Rabung certificates, at lengths 17, 18 and 19. They give nine direct inequalities for $17 \leq k \leq 25$; the final six follow from the length-19 certificate by a fixed-prime monotonicity lemma. In particular,

$$W(3, 17) > 31\,385\,622\,833,$$

using $p = 1\,961\,601\,427$. This is a factor 2.02 above the earlier $15\,509\,557\,937$ listed by Monroe. The five inequalities for $17 \leq k \leq 21$ improve the prior comparators identified in the source review updated on 26 September 2026; Table 1 gives their values.

A new direct certificate need not give a better general bound. For $22 \leq k \leq 25$, our certificates improve the Rabung rows located in the review, but stronger bounds already follow from other constructions. A stronger published statement of Landman and Robertson has an unresolved proof status in our audit. We display its consequences separately and include it among prior comparators: the five improvements at lengths 17–21 remain, whereas it dominates the admitted table at lengths 24–28. This is a comparison with a dated, explicit corpus, not an assertion of exhaustive global priority.

We also recheck four two-colour certificates from the archived second phase of Monroe’s distributed project [16], retaining the credit of that project and its volunteers. Together with the nine three-colour rows, they form the thirteen inequalities of Theorem 2.1. Only seven triples are primitive; the archived rechecks deliberately retain the redundant consequences.

1.2 From a short certificate to a finite comparison

Multiplication by a nonzero residue permutes the colour classes of a power-residue colouring. After such a dilation, an arithmetic progression becomes a consecutive string. Rabung’s theorem reduces the large-colouring problem to a run-length test and a condition at the multiples of p [21]. A qualifying prime gives $W(r, k) > (k - 1)p + 1$. We prove the equivalence of the implemented boundary test with the published criterion and provide a stand-alone CPU checker whose source and arithmetic are separate from the GPU search. The small example in Section 3.2 illustrates the reduction before the proof.

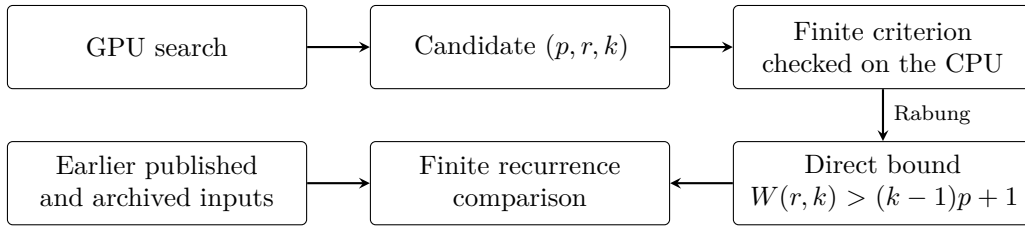


Figure 1: The two steps after discovery. The candidate can be checked without replaying the search, and its bound is compared with consequences of earlier inputs. Campaign coverage and the empirical intensity analysis are separate from these pointwise implications.

Earlier bounds must then be compared under known recurrences. For example, Blankenship, Cummings and Taranchuk [2, Theorem 2.1] prove

$$W(r, k) > q(W(r - \lceil r/q \rceil, k) - 1) \quad (1)$$

for a prime $q \leq k$. For three colours, an earlier strict bound $W(2, k) > B$ gives $W(3, k) > q_k B$, where q_k is the largest prime at most k . This explains why a larger Rabung prime need not yield the strongest general bound. Our finite comparison also includes length monotonicity and the registered Xu operation [18, 23], with exact integers and a provenance path for every retained value. No winning three-colour row depends on Xu.

1.3 Relation to earlier work

Computational power-residue constructions and cyclic-zipper extensions have been developed by Herwig, Heule, van Lambalgen and van Maaren [6], Rabung and Lotts [20], Liang, Xu, Shao and Xiu [12], and Monroe [13, 14, 18]. Publishing a prime together with checking code has antecedents in this literature. Our contribution is the three new primitive certificates, their

separate verification, and their explicit comparison with consequences of the identified prior inputs. Certification is not presented as a new principle.

Berlekamp’s construction supplies a stronger admitted bound at length 24, inherited at length 25 [1]. The stronger Landman–Robertson statement requires the separate treatment just described [8, 10, 11]. The recent superexponential three-colour result of Fox and Hunter [4] and the asymptotic two-colour bound $w(k) \geq (1 - o(1))k2^{k-1}$ of Campos, Fox and Schildkraut [3] address different asymptotic questions. In the version-pinned sources examined, neither supplies a stronger explicit finite entry for our window. Gasarch and Haeupler’s explicit family $W(3, k) \geq 3^{k-1}/(ek)$ [5, Note 1.7(2)] is retained in the registry and is dominated there. Kozik and Shabanov [9, Theorem 2] give $W(r, k) \geq \beta r^{k-1}$ for an absolute $\beta > 0$; no finite improvement from that theorem is asserted here. Hunter’s exponential improvement begins at five colours [7] and does not change the three-colour table.

1.4 Reading guide and evidence boundary

Section 2 gives the numerical results and their comparison. Section 3 explains the construction, proves the boundary-test equivalence and describes the checker. Section 4 explains the discovery and verification protocol. The search covered $[9.7 \times 10^8, 2 \times 10^9)$ in 515 chunks; its coverage evidence has a separate role from the validity of any one certificate.

A reader interested in the inequalities can read the results and the criterion proof first. The source catalogue, campaign audit and execution identities remain available in the appendices and accompanying package. Appendices A–C compare three fixed search-intensity forecasts on one run-valid (3, 17) trajectory. That empirical comparison neither proves a density law nor enters the lower-bound proof. No independently administered external replication has been performed; the preserved local computations and their limits are stated throughout.

2 Main results and exact scope

The computation gives the thirteen direct inequalities stated below: nine at three colours and four re-verifications of earlier two-colour certificates. Their supporting evidence consists of archived executions bound to exact source code, binaries and raw output. Checking the archive and rebuilding this paper requires no repetition of the GPU campaign.

Theorem 2.1 (Certified Rabung bounds). *For every triple (p, r, k) in Table 7, p is prime, $p \equiv 1 \pmod{r}$, $p > k$, and the power-residue colouring satisfies conditions $(a) \wedge (B^*)$. Consequently*

$$W(r, k) > (k - 1)p + 1.$$

In particular, all thirteen direct inequalities registered in `audit/claims.json` hold.

Proof. For each row, the archived execution of the stand-alone verifier establishes primality, $p \equiv 1 \pmod{r}$, $p > k$, and conditions $(a) \wedge (B^*)$. Proposition 3.3 identifies the operational condition with Rabung’s boundary criterion, and Rabung’s theorem then gives the displayed inequality. The build audit checks the exact set of thirteen manifests and their bound source, executable and output hashes before including the table; it does not repeat those long executions. $\square$

The full execution-identity table is provided in Appendix D.

2.1 Three colours: direct and recurrence-closed bounds

The historical scan produced nine direct Rabung rows, all now accepted by the final manifest audit. They involve only three distinct primes. By Lemma 3.2, the certificate (1 999 999 927, 3, 19) implies all six rows with the same prime and $20 \leq k \leq 25$. Thus only seven triples in the thirteen-row theorem are mathematically primitive: three at three colours and the four two-colour triples below.

The evidence package nevertheless re-evaluates the complete criterion at all thirteen registered triples as deliberate audit redundancy; the six repeated checks are neither mathematically necessary nor counted as independent implementation paths.

The headline value

$$W(3, 17) > 31\,385\,622\,833$$

comes from $p = 1\,961\,601\,427$ and improves Monroe’s published $15\,509\,557\,937$ by a factor 2.02 [18]. A direct certificate is not automatically a general record: a published bound in another colour column may imply a stronger result through a recurrence.

Table 1 states the five improvements against the earlier comparators identified in our review. For this comparison the new three-colour claims are removed from the finite closure, and the published Landman–Robertson statement is included as a conservative comparator even though its proof was not verified here. Thus the comparison does not obtain an artificial gain by discarding that statement.

k	new strict bound	prior comparator	comparator source
17	31 385 622 833	15 509 557 937	Monroe tally
18	33 999 912 468	16 479 723 552	Monroe tally
19	35 999 998 687	17 449 137 523	Monroe tally
20	37 999 998 614	22 082 967 854	Landman–Robertson*
21	39 999 998 541	26 783 744 079	BCT from two colours

Table 1: The five improvements over prior comparators identified through 26 September 2026. Every numerical entry B uses the strict convention $W(3, k) > B$. *The published Landman–Robertson statement is included for a conservative comparison; its proof was not verified in our review. The finite prior closure excludes the new three-colour claims.

Let q_k denote the largest prime not exceeding k . From Blankenship–Cummings–Taranchuk [2, Theorem 2.1],

$$W(3, k) > q_k(W(2, k) - 1).$$

Their theorem permits every prime $q \leq k$. For three colours, $q = 2$ reduces to the one-colour case, while every $q \geq 3$ uses the same two-colour input; choosing the largest such prime therefore loses no BCT consequence for the three-colour outputs considered here. If the registered input is the strict integer bound $W(2, k) > B$, then $W(2, k) - 1 \geq B$, and the generated consequence is exactly

$$W(3, k) > q_k B, \tag{2}$$

with no off-by-one ambiguity.

For completeness, write $WR(s, k)$ for Xu’s ring van der Waerden number: the least N such that, for every $n \geq N$, every s -colouring of $\mathbb{Z}/n\mathbb{Z}$ has a monochromatic k -term arithmetic progression. The prime-witness specialization of Xu’s concatenation recurrence used in Monroe’s later computation [18, 23] has $k \geq 3$, $s \geq 2$, $t \geq 1$, and $5 \leq n < WR(s, k)$ whose least prime divisor exceeds k ,

$$W(st, k) \geq n(W(t, k) - 1) + 1. \tag{3}$$

The original full text of Xu was not obtained in the latest review; the specialization used here was read in Monroe’s account, and its numerical preconditions are checked for every registered witness. No winning three-colour row depends on Xu. Our registry stores strict integer bounds. Every ring witness R used here has least prime divisor greater than k , a precondition checked by the closure tests. Hence inputs $WR(s, k) > R$ and $W(t, k) > B$ permit $n = R$ and give exactly $W(st, k) > RB$, with no rounding or off-by-one convention left implicit.

The baseline program iterates elementary length monotonicity, the registered BCT specialization and (3) to a fixed point. Its closure state covers $12 \leq k \leq 28$ and at most four colours; the reported comparison window remains $17 \leq k \leq 28$. Published direct and Berlekamp seeds

are loaded from $k = 12$ so earlier monotone consequences are not truncated. At each step it retains exact integer bounds and a full provenance path. Candidates are compared first by bound; numerical ties use the deterministic order direct Rabung, published or archived seed, length monotonicity, BCT, Xu, one-colour exact, and finally the hash-derived node identifier. These tie-breaks affect provenance only. Since all three operators are monotone in their input bounds, a dominated parent cannot improve a descendant. Iteration stops when no winning node changes.

The seed registry and its dated source catalogue are detailed in Appendix E. Published and archived predecessors are loaded from length 12, so consequences entering the reported window from earlier lengths are retained. Throughout this paper, an *admitted bound* means a consequence of those explicitly registered inputs and operations. The separately displayed Landman–Robertson statement has a different status.

Berlekamp writes $W(K, t)$ for K colours and progressions of length $t + 1$. Specializing his Theorem 1 to $K = 3, t = 23$ gives

$$W(3, 24) > \frac{23(3^{23} - 1)}{2} = 1\,082\,646\,556\,499, \quad (4)$$

after checking the theorem’s divisor conditions [1]. Length monotonicity gives the same strict bound for $W(3, 25)$. The registry also contains the sixteen directly dominated specializations, so this winning case is not an isolated post hoc insertion.

Landman and Robertson state, without an accompanying proof, that for primes $p \geq 5$ and q ,

$$w(p + 1; q) \geq p(q^p - 1) + 1$$

in their first and second editions [10, Theorem 2.19, p. 38] [11, Theorem 2.21, p. 43]. For $q = 3$, its specializations at $p = 19$ and $p = 23$ give respectively

$$W(3, 20) > 22\,082\,967\,854, \quad W(3, 24) > 2\,165\,293\,112\,998.$$

The latter would dominate the admitted comparison through $k = 28$ by length monotonicity. Berlekamp’s primary construction proves the general factor $1/(q - 1)$, with its stronger extension confined to $q = 2$, and Johannson’s detailed reconstruction reports that the proposed general adaptation could not be confirmed [8, pp. 88–90]. This does not disprove the textbook statement. It is excluded from the numerical registry of admitted constructions and certificates, but retained as a published statement with unresolved proof status in our audit. Table 2 makes its numerical effect explicit. In particular, the admitted values for $24 \leq k \leq 28$ must not be described without this qualification as the best published bounds. The $k = 20$ specialization is already included in Table 1 and does not remove that improvement.

k	best admitted consequence	published unresolved statement
20	37 999 998 614	22 082 967 854
24	1 082 646 556 499	2 165 293 112 998
25	1 082 646 556 499	2 165 293 112 998
26	1 221 912 599 848	2 165 293 112 998
27	2 068 976 009 617	2 165 293 112 998
28	2 159 051 058 266	2 165 293 112 998

Table 2: Two distinct comparison levels. The middle column uses the checked seed registry, including this work. The last column records the numerical consequences of the stronger published Landman–Robertson statement and length monotonicity; these are not certified by this paper.

The arXiv version history dates Monroe’s earlier $W(3, 17) > 15\,509\,557\,937$ and prime 969 347 371 to version 4 of 6 November 2017; versions 1–3 do not contain them [14]. Exact-number, prime, publisher, preprint, citation-chain, repository and web archive searches were performed. No earlier public source was located for the three primitive direct triples used here at

$k = 17, 18, 19$ in the review updated on 26 September 2026. This bounded search result does not establish global priority. We therefore describe those certificates as new to the audited corpus rather than assert an unqualified first discovery. The search methods, version pins, admitted and excluded findings, and exact cutoff are machine-readable in `publication/bibliography-audit.json`.

k	direct p	direct bound	winning admitted bound	method
17	1 961 601 427	31 385 622 833	31 385 622 833	direct Rabung
18	1 999 994 851	33 999 912 468	33 999 912 468	direct Rabung
19	1 999 999 927	35 999 998 687	35 999 998 687	direct Rabung
20	1 999 999 927	37 999 998 614	37 999 998 614	direct Rabung
21	1 999 999 927	39 999 998 541	39 999 998 541	direct Rabung
22	1 999 999 927	41 999 998 468	49 058 715 046	BCT, $q = 19$ (published $W(2, k)$)
23	1 999 999 927	43 999 998 395	142 741 265 701	BCT, $q = 23$ (published $W(2, k)$)
24	1 999 999 927	45 999 998 322	1 082 646 556 499	Berlekamp (1968)
25	1 999 999 927	47 999 998 249	1 082 646 556 499	Berlekamp + length monotonicity
26	–	–	1 221 912 599 848	BCT, $q = 23$ (archived input)
27	–	–	2 068 976 009 617	BCT, $q = 23$ (archived input)
28	–	–	2 159 051 058 266	BCT, $q = 23$ (archived input)

Table 3: Direct three-colour Rabung certificates and the winning lower bounds obtained after closing the audited, admitted baselines under length monotonicity and the published Blankenship–Cummings–Taranchuk and Xu recurrences. A dash means that the GPU campaign made no direct three-colour claim for that length. For $k = 22, 23, 24, 25$ the direct certificates are the strongest direct Rabung bounds located in the audited corpus but do not win the admitted comparison. Every entry is generated with a machine-readable provenance path in `audit/generated/bounds_closure.json`. The stronger published Landman–Robertson statement is listed separately in Table 2; this is not an unqualified table of best published bounds.

Corollary 2.2 (Recurrence-closed consequences). *Within the frozen seed set and finite scope just specified, closing under the length-monotonicity, Blankenship–Cummings–Taranchuk and Xu operations gives the winning strict bounds in Table 3. In particular, the certified direct rows win for $17 \leq k \leq 21$, while for $22 \leq k \leq 25$ they are dominated by the displayed admitted consequences.*

Thus the certified rows for $17 \leq k \leq 21$ give the strongest bounds located in the registered, recurrence-closed source corpus. For $22 \leq k \leq 25$ the direct rows improve the direct Rabung family but do not win the admitted comparison. BCT consequences of published two-colour values win at $k = 22, 23$; Berlekamp wins at $k = 24, 25$. No winning three-colour row in this finite scope uses Xu’s recurrence; it is retained because the comparison claim includes all registered operations.

2.2 Archived two-colour inputs and recurrence consequences

After the scan, we located a table from a second, unpublished phase of Daniel Monroe’s Vd-wnumbers.org BOINC project (2021, two colours only) [16]. The earliest public evidence located in our audit attributes the four values to that project and its volunteer contributors. A later author-owned project page reports retrospectively that this phase pushed the checked range from one billion to four billion [17]; the largest prime actually listed in the recovered table is 3 478 685 143. We use our checker only to test whether the four displayed triples satisfy Rabung’s criterion.

For $k = 25$, even Monroe’s published input would give

$$W(2, 25) > 23\,003\,662\,489 \implies W(3, 25) > 529\,084\,237\,247.$$

The archived input accepted in the final manifest audit, $W(2, 25) > 27\,333\,622\,969$, strengthens that BCT consequence to $W(3, 25) > 628\,673\,328\,287$, but Berlekamp’s monotone consequence is stronger. The winning admitted bounds for this four-length block are therefore

k	prime p	$W(2, k) >$	method and provenance
25	1 138 900 957	27 333 622 969	phase-2 table; accepted final manifest
26	2 125 065 391	53 126 634 776	phase-2 table; accepted final manifest
27	3 459 826 103	89 955 478 679	phase-2 table; accepted manifest and no-Montgomery witness
28	3 476 732 783	93 871 785 142	phase-2 table; accepted manifest and no-Montgomery witness

Table 4: Two-colour succinct Rabung certificates used as recurrence inputs. The method column separates dated phase-2 attribution from the separate-source recheck recorded in the accepted final manifests.

$$W(3, 25) > 1\,082\,646\,556\,499$$

$$W(3, 26) > 1\,221\,912\,599\,848$$

$$W(3, 27) > 2\,068\,976\,009\,617$$

$$W(3, 28) > 2\,159\,051\,058\,266$$

The first line comes from Berlekamp and length monotonicity; the last three are BCT consequences of the corresponding phase-2 two-colour inputs. They are not additional GPU discoveries. Xu’s recurrence is included in the automatic closure but does not improve any winner. The accompanying data and checks are described in Section 6.

For $k = 25$ the authenticated campaign record identifies the same last candidate in $[9.7 \times 10^8, 2 \times 10^9]$ as the archived table, and the ordered-prime audit agrees on all 515 chunks. For $k = 26, 27, 28$ our campaign-range candidates are weaker, and the displayed archive primes are accepted directly by the final manifests. None of this establishes that Monroe’s phase exhaustively tested every prime up to four billion: that is an attributed archival statement, not a consequence of the recovered table.

The final source file is internally dated 18 August 2021, whereas the first surviving Internet Archive capture containing the exact final $k = 27, 28$ values is dated 28 April 2022 [16]. Earlier surviving captures establish the final $k = 25$ value by 4 March 2021 and the final $k = 26$ value by 5 May 2021. We report source-internal and independently preserved dates separately. The release records all five capture URLs, hashes, and contents in `publication/baseline-provenance.json`; it does not redistribute the third-party table absent permission. The defensible claim is a separately auditable re-verification with a public stand-alone checker and complete logs—not first discovery of the primes and not a gap-free audit of the historical scan.

3 The Rabung criterion and certificates

3.1 Power-residue colourings

The certification argument has three layers. Rabung’s theorem turns a finite criterion into a lower bound; Proposition 3.3 proves the equivalent operational form used here; and a stand-alone, separate-source checker re-establishes that criterion for each reported triple. The GPU search of Section 4 discovers candidate triples but is not a premise of this implication.

An *explicit certificate* for $W(r, k) > n$ is an r -colouring of $\{1, \dots, n\}$ with no monochromatic k -term progression. The large claims instead use a *succinct Rabung certificate* (p, r, k) , from which the checker reconstructs the hypotheses of the construction theorem. It does not enumerate every progression in the resulting colouring.

Let p be a prime with $p \equiv 1 \pmod{r}$ and $p > k$, let g be a primitive root modulo p , and let

$\text{ind}(x)$ denote the discrete logarithm of x to base g . The *power-residue colouring* is

$$c(x) = \text{ind}(x) \bmod r, \quad x \in \{1, \dots, p-1\},$$

i.e. the partition of $\mathbb{Z}_p^*$ into the r classes of r -th power residues. Its key property is multiplicativity: $c(xy) = c(x) + c(y) \pmod{r}$. Consequently, dilating a progression $\{a, a+d, \dots, a+(k-1)d\}$ with $p \nmid d$ by $u = d^{-1} \bmod p$ shifts all colours by the constant $c(u)$ and maps the progression to k consecutive residues modulo p . The search for monochromatic progressions therefore reduces to a linear scan of the run lengths of the sequence $c(1), c(2), \dots, c(p-1)$, plus a boundary condition at the multiples of p ; this reduction from an $O(p^2/k)$ test to an $O(p)$ test per prime is what makes a mass scan possible.

3.2 Rabung's theorem and the operational criterion

Rabung [21] proved that the extension ϑ' of the power-residue colouring to $[0, (k-1)p]$, with the multiples of p colored by any non-constant assignment, is free of monochromatic k -term progressions *if and only if*

- (a) no k consecutive positions in $[1, p-1]$ receive the same colour (max run $< k$); and
- (b) a boundary condition holds. Put $\varepsilon := c(-1) = c(p-1) \in \{0, r/2\}$, where the second value is possible only for even r . In Rabung's multiplicative notation the corresponding sign is $+1$ when $\varepsilon = 0$ and -1 when $\varepsilon = r/2$. If $\varepsilon = 0$ the integers $1, 2, \dots, \lceil (k-1)/2 \rceil$ must not all lie in the same class, while if $\varepsilon = r/2$ the same is required of $1, 2, \dots, k-1$. A *singleton* counts as “all in the same class”, and so fails (b).

If r is odd, $(p-1)/r$ is even and hence $\varepsilon = 0$ necessarily.

Remark 3.1. Both details of (b) matter. Rabung's bracket here denotes the ceiling. Replacing it by a floor, or treating a singleton as vacuously satisfying the condition, yields a criterion that is *not* equivalent to Rabung's. For $p = 5$, $r = 2$, $k = 4$, condition (a) holds and the ceiling correctly accepts the boundary test, whereas the floor rejects it. For $p = 5$, $r = 2$, $k = 3$, treating the singleton as acceptable gives a false acceptance of the complete criterion. The ceiling and floor coincide only for odd k . Both failure modes occurred during development and were detected by comparison with the finite oracle described below.

Since $[0, (k-1)p]$ contains $(k-1)p + 1$ integers, a prime satisfying (a) $\wedge$ (b) yields

$$W(r, k) > (k-1)p + 1. \tag{5}$$

We call such a prime *valid* for (r, k) .

Our implementations test an equivalent form of (b), denoted (B^*) , which is uniform in the parity of k and the value of ε : for every shift $j \in \{0, \dots, k-1\}$, the $k-1$ character positions $\{-j, \dots, -1, 1, \dots, k-1-j\}$ of a k -window centered on a multiple of p must not all receive the same colour. The equivalence rests on the multiplicative structure: if a single shifted window is monochromatic, its dilates realize every colour at the central position (the congruence $p \equiv 1 \pmod{r}$ makes c a surjective homomorphism to $\mathbb{Z}/r\mathbb{Z}$), so the joker position cannot be colored at all; otherwise any non-constant colouring of the multiples of p works. Proposition 3.3 gives the mathematical proof; the finite comparison below audits the implementation rather than replacing that proof.

A small example. Take $(p, r, k) = (5, 2, 4)$ and primitive root $g = 2$. The nonzero residue colours are $(c(1), c(2), c(3), c(4)) = (0, 1, 1, 0)$. The longest run is $2 < 4$, and the boundary interval $\{1, 2\}$ contains both colours. Thus the criterion gives $W(2, 4) > 16$: the colouring on $[0, 15]$ consists of three repeated nonzero residue blocks separated by four joker positions. Every non-constant

assignment to those jokers works. This example illustrates the construction; it is not asserted to be the best bound at this small length. It also shows why replacing the ceiling by a floor would discard a valid certificate.

Lemma 3.2 (Monotonicity at fixed prime). *Let $r \geq 2$, let $3 \leq k \leq \ell < p$, and let $p \equiv 1 \pmod{r}$ be prime. If p satisfies $(a) \wedge (B^*)$ for (r, k) , then it satisfies the same criterion for (r, ℓ) . Consequently*

$$W(r, \ell) > (\ell - 1)p + 1.$$

Proof. It is enough to pass from k to $k + 1$. A monochromatic run of length $k + 1$ would contain one of length k , so condition (a) persists. Write

$$S_j^{(m)} = \{-j, \dots, -1\} \cup \{1, \dots, m - 1 - j\}, \quad 0 \leq j \leq m - 1.$$

For $0 \leq j \leq k - 1$ we have $S_j^{(k)} \subset S_j^{(k+1)}$, and $S_{k-1}^{(k)} \subset S_k^{(k+1)}$. Thus every boundary string for length $k + 1$ contains one already known to be non-monochromatic, so (B^*) persists. Iteration gives the claim. $\square$

3.3 Equivalence of the operational criterion

The scan tests the criterion $(a) \wedge (B^*)$ of Section 3 rather than Rabung's $(a) \wedge (b)$ verbatim. The two are equivalent, and since every bound in this paper rests on that equivalence we prove it here rather than rely on the exhaustive machine comparison of Section 3, which we regard as a check on the *implementation*, not as a substitute for a proof.

Setting. Let $p \equiv 1 \pmod{r}$ with $p > k \geq 3$, let g be a primitive root, and colour $x \in [1, p - 1]$ by $c(x) = \text{ind}_g(x) \bmod r$; this is the power-residue colouring, and $c(xy) = c(x) + c(y)$. Extend it to $[0, (k - 1)p]$ by reducing modulo p and colouring the multiples of p (the *jokers*) by an arbitrary map $\vartheta' : \{0, p, \dots, (k - 1)p\} \rightarrow \mathbb{Z}_r$. Put $\varepsilon := c(-1) = c(p - 1)$, which is 0 if -1 is an r -th power residue and $r/2$ otherwise (the latter requires r even).

An arithmetic progression of length k and common difference d is monochromatic iff its terms all receive one colour. Progressions avoiding the jokers are handled by (a): by complete multiplicativity of c , the progression $a, a + d, \dots, a + (k - 1)d$ reduces, after dividing by d (which permutes the classes by the constant $c(d)$), to the window $\{a/d, a/d + 1, \dots, a/d + k - 1\}$ of k consecutive residues. Hence *some* joker-free progression is monochromatic iff *some* window of k consecutive residues in $[1, p - 1]$ is, which is exactly the negation of (a).

The joker condition. It remains to treat the progressions that meet a joker. First bound the common difference: a k -term progression inside $[0, (k - 1)p]$ has $(k - 1)d \leq (k - 1)p$, so $1 \leq d \leq p$, and the case $d = p$ occurs for exactly one progression, namely $0, p, 2p, \dots, (k - 1)p$, which consists *entirely* of jokers. That progression is monochromatic if and only if ϑ' is constant—it is precisely the reason Rabung's statement requires a *non-constant* joker colouring, and any non-constant ϑ' breaks it. For the remaining case $0 < d < p$: since the progression has $k < p$ terms, two of them congruent to 0 modulo p would force $p \mid (i' - i)d$ with $0 < |i' - i| < k < p$ and $p \nmid d$, which is impossible; so the progression meets at most one multiple of p . Say it is the $(j + 1)$ -st term, $0 \leq j \leq k - 1$. Dividing by d as before, the progression becomes the *shifted string*

$$S_j = \{-j, \dots, -1\} \cup \{1, \dots, k - 1 - j\} \pmod{p},$$

centred on the joker, together with the joker itself.

Proposition 3.3. *Let $r \geq 2$, let $p \equiv 1 \pmod{r}$ be prime, and let $3 \leq k < p$. The colouring above is free of monochromatic k -term progressions for some choice of ϑ' if and only if $(a) \wedge (B^*)$ holds, where*

$$(B^*) : \quad \text{no shifted string } S_j \text{ } (0 \leq j \leq k - 1) \text{ is monochromatic.}$$

Moreover, when $(a) \wedge (B^*)$ holds, every non-constant ϑ' works; and when it fails, no ϑ' works. In particular the choice of ϑ' is immaterial, as asserted by Rabung.

Proof. Suppose some S_j is monochromatic, of colour w say.

The lift. We first check that every residue class of d is realised by a genuine progression inside $[0, (k-1)p]$ through a joker. Fix a representative $d \in [1, p-1]$ and take the joker to be jp and the first term to be

$$a = j(p-d).$$

Then $a + jd = jp$, so the $(j+1)$ -st term is the joker; the first term $a = j(p-d) \geq 0$ since $d < p$; and the last term is

$$a + (k-1)d = jp + (k-1-j)d \leq jp + (k-1-j)(p-1) \leq (k-1)p,$$

with equality in the last step only at $j = k-1$, where that last term is the joker jp itself; so all k terms lie in $[0, (k-1)p]$. The non-joker terms are $jp + id$ for $i \in S_j$, whose residues are $id \bmod p$; hence their colours are $c(i) + c(d) = w + c(d)$.

The all-or-nothing mechanism. As d ranges over $(\mathbb{Z}/p)^*$, the value $w + c(d)$ ranges over *all* of $\mathbb{Z}_r$ (because c is onto), and by the lift each such value is attained by an admissible progression through the *same, fixed* joker jp . Hence for each colour $u \in \mathbb{Z}_r$ there is a progression through jp whose other $k-1$ terms are all of colour u ; assigning $\vartheta'(jp) = u$ makes it monochromatic. Every value of ϑ' at the single joker jp is thus forbidden, so no choice of ϑ' —constant or not—avoids a monochromatic progression. Conversely, if no S_j is monochromatic then no progression with $d < p$ through a joker has all its other $k-1$ terms of a single colour, so no assignment of ϑ' can complete one; the only remaining progression is the all-joker one ($d = p$), which any *non-constant* ϑ' breaks; combined with (a) for the joker-free progressions, every non-constant ϑ' works. The $d = p$ case is exactly what consumes the non-constancy hypothesis. Thus $(a) \wedge (B^*)$ is all-or-nothing over non-constant ϑ' , which is why Rabung may state it without reference to the joker colouring. $\square$

Reduction of (B^*) to a single interval. Since $c(-x) = c(x) + \varepsilon$, the string S_j is monochromatic iff the two intervals $\{1, \dots, j\}$ and $\{1, \dots, k-1-j\}$ are each monochromatic and their colours agree after the shift by ε .

Case $\varepsilon = 0$. Then $c(-x) = c(x)$, so S_j is monochromatic iff the interval $\{1, \dots, \max(j, k-1-j)\}$ is. Minimising $\max(j, k-1-j)$ over $0 \leq j \leq k-1$ gives $\lceil (k-1)/2 \rceil$, attained at $j = \lfloor (k-1)/2 \rfloor$. Hence

$$(B^*) \iff \{1, 2, \dots, \lceil (k-1)/2 \rceil\} \text{ is not monochromatic.}$$

A one-element interval *is* monochromatic, and therefore fails the condition; under the standing assumption $k \geq 3$ this is the case $k = 3$, and it is the reason a singleton must be counted as “all in the same class”.

Case $\varepsilon = r/2$ (r even). Now $c(-x) = c(x) + r/2 \neq c(x)$, so a two-sided string can never be monochromatic: only the one-sided strings $j = 0$ and $j = k-1$ survive, both giving the interval $\{1, \dots, k-1\}$. Hence

$$(B^*) \iff \{1, 2, \dots, k-1\} \text{ is not monochromatic,}$$

Unlike the length- k prohibition of (a), this is a condition on an interval of length $k-1$, and is therefore *not* implied by (a): it is a genuine additional constraint, and on a longer interval than in the previous case.

Remark 3.4. Both refinements matter. Replacing $\lceil \cdot \rceil$ by $\lfloor \cdot \rfloor$, or treating a singleton as vacuously satisfying the condition, gives a boundary condition that is not equivalent to (b). Counterexamples occur already at $p = 5$, $r = 2$, $k = 4$ for the floor variant and at $p = 5$, $r = 2$, $k = 3$ for the singleton convention. In both examples the error changes the verdict of the complete criterion. At $p = 13$, $r = 2$, $k = 4$, the same boundary discrepancy is masked by failure of (a).

This proposition is the mathematical bridge used for every one of the thirteen registered triples: once a verifier establishes $(a) \wedge (B^*)$, Rabung’s theorem yields the corresponding direct inequality.

3.4 Finite oracle audit of the implementation

We tested the implementation of $(a) \wedge (B^*)$ against a frozen reference oracle that explicitly builds the colouring of $[1, (k-1)p+1]$ and exhaustively checks all k -term progressions. One bookkeeping detail matters: Rabung’s construction lives on $[0, (k-1)p]$, and the oracle works on its translate by one—it colours x by the class of $(x-1) \bmod p$ and places the jokers at $x \equiv 1 \pmod{p}$ —so the two objects are literally the same colouring up to relabeling of positions, and freedom from monochromatic progressions is translation-invariant. A sweep over all primes $p < 10^4$ with $p \equiv 1 \pmod{r}$, $p > k$, $r \in [2, 6]$, $k \in [3, 8]$ — 20 170 cases, of which 4 499 were valid according to the criterion — reported zero disagreements. A fresh staging replay reproduced those counts and its exact source identity, environment, and output streams are archived with this repository. This larger oracle sweep is historical evidence; the final CPU suite separately repeats the small oracle cases and checks the retained record. A stronger test enumerates all r^k colourings of the joker positions ($p < 150$, $r \leq 4$, $k \leq 5$) and confirms both directions of the equivalence: $(a) \wedge (b)$ holds if and only if some valid extension exists, and when it holds every non-constant joker colouring is valid. We emphasize the epistemic status: Rabung’s theorem is proved in [21]; the agreement of our operational form with the explicit oracle is exhaustively machine-checked on the stated domain.

3.5 Certificates and verification regimes

For the bounds announced in Section 2 the certificate length $(k-1)p+1$ is of order 3×10^{10} to 10^{11} , so running the naive quadratic oracle on the explicit certificate is not feasible. Our verification policy therefore distinguishes two regimes. On the finite validation domain actually reported above ($p < 10^4$, $k \leq 8$, hence certificate length below 7×10^4), the explicit certificate was checked directly by the reference oracle. We make no general feasibility claim for length 10^7 : a successful naive oracle must inspect

$$\sum_{d=1}^{\lfloor (n-1)/(k-1) \rfloor} (n - (k-1)d)$$

progressions. For the large primes, the proof of (5) is Rabung’s theorem together with conditions $(a) \wedge (b)$, and *these* are what we verify redundantly: the mirror, full-interval, and CPU-walk paths, followed by separate CPU implementations of the full criterion. Section 4.2 records which paths share source code or arithmetic; cross-agreement is not described as statistical or implementation independence.

At the large sizes of Section 2, distributing the full colouring would be neither feasible nor useful. The reproducibility package therefore distributes the triple (p, r, k) , from which the stand-alone, separate-source verifier `verify_claim` re-derives every required hypothesis. Its protocol is:

1. check that p is prime by deterministic Miller–Rabin on the first twelve prime bases 2, 3, 5, ..., 37. Sorenson and Webster’s Theorem 1.1 gives

$$\psi_{12} = 318\,665\,857\,834\,031\,151\,167\,461 > 2^{64},$$

so this is exact through the unsigned 64-bit range [22], far beyond the $p < 2^{32}$ actually used;

2. check the preconditions $p \equiv 1 \pmod{r}$ and $p > k$;
3. *find* a primitive root g by trial, certifying primitivity against the full trial-division factorization of $p-1$ (nothing about g is taken as input);
4. build the residue classes by modular exponentiation— $x^{(p-1)/r}$ compared against the powers of $\zeta = g^{(p-1)/r}$ —using neither the discrete-logarithm walk nor the GPU pipeline;

5. test $(a) \wedge (B^*)$ and print ACCEPT or REJECT.

For the fixed small colour counts used here, the checker takes $O(p \log p)$ modular-arithmetic steps and $O(p)$ memory. It stores one byte per residue at $r = 2, 3$: the largest certificate therefore requires about 3.48 GB for the colour array alone, before auxiliary memory. This is the main practical cost of re-evaluating a large triple. The archived executions already establish the finite checks used here; validating their identities and rebuilding the paper are much lighter operations. No standardized runtime benchmark is claimed. A streaming implementation could reduce memory, but it is not the implementation whose executions are reported.

4 GPU discovery campaign

The campaign has one logical role: it generates candidate triples (p, r, k) . No lower bound follows from a scanner verdict alone. A retained candidate becomes a numerical claim only after the criterion is re-established through the verification protocol below. The authenticated coverage-evidence package supports the coverage and density statements, but is not needed for the pointwise validity of an accepted certificate.

All qualification runs and claim rechecks reported here were run on one author-controlled PC. “Separate-source” denotes separation of implementation and arithmetic paths from the GPU scanner; it does not denote an independently administered replication, and no external replication has been performed.

4.1 Fused character evaluation

The scan kernel exploits the fact that the additive order of positions is free: we iterate $x = 1, 2, \dots, p-1$ directly and evaluate the colour of x on the fly by modular exponentiation, fused with the run-length bookkeeping. Nothing is sorted and no per-element value is materialized in memory. A single pass evaluates $y = x^{(p-1)/m}$ with $m = \text{lcm}\{r \leq 9 : r \mid p-1\} \leq 2520$; a table lookup of y among the m -th roots of unity gives $\text{ind}(x) \bmod m$, hence the colour for every modulus $r \leq 9$ simultaneously. Runs are summarized per segment by (prefix, suffix, best interior) triples combined with an associative, order-respecting reduction.

A second symmetry halves the work. Since $c(p-x) = c(x) + c(-1)$, the second half of the sequence is the reversal of the first half shifted by the constant $c(-1)$; it suffices to scan $x = 1, \dots, (p-1)/2$ and to splice the two halves at the center. We refer to this half-interval variant as the mirror scan.

4.2 Redundant and separate-source validation paths

The retained CUDA qualification compares the mirror, full-interval, sort-based and CPU-walk paths on small primes, tests the boundary condition by two constructions, and exercises the high-range modular arithmetic. Its raw logs and source identities are preserved with the qualification manifest. The current CPU suite also tests the order-respecting run reduction. The historical development exposed why this matters: run summaries compose associatively but not commutatively, so a reduction that reorders them can produce an incorrect profile. The archived scanner uses the repaired ordered reduction. Agreement between implementations is interpreted according to the components they share, not as independent external replication.

The main paths have the following dependency structure; agreement between paths is interpreted in light of their shared source and arithmetic.

4.3 Claim-verification protocol

The computational source of truth registers exactly thirteen direct Rabung claims: nine three-colour triples for $17 \leq k \leq 25$ and four two-colour triples for $25 \leq k \leq 28$. For each triple (p, r, k)

path	language	modular arithmetic	class/run construction	shared components
mirror V2a	CUDA C++	GPU Montgomery	character, half interval	same source and helpers as full B7
full B7	CUDA C++	GPU Montgomery	character, full interval	same source and helpers as V2a
sort-based GPU	CUDA C++	GPU Montgomery	sort then run scan	earlier GPU design; shares GPU Montgomery arithmetic
CPU walk	C++ host	native 64-bit walk	discrete logarithm, sequential	same executable and host primitive-root code
CPU criterion	C++	separate 128-bit path	separate walk and (B*)	separate source and binary
<code>verify_claim</code>	C++	128-bit exponentiation	roots of unity and full criterion	strongest source/arithmetic separation
high-range witness	C	no Montgomery	full sequential walk	separate arithmetic; tests (a), not (B*)

Table 5: Dependency matrix for the verification paths. V2a and B7 are correlated redundant paths. The principal source- or arithmetic-separated checks are the separate CPU criterion, `verify_claim`, and, for the high-range arithmetic defect, the Montgomery-free witness.

the release protocol requires:

1. two executions of the structured triplet {mirror scan, full scan, CPU walk}, with internal profile equality and equality between executions;
2. two acceptances by the separate full CPU criterion;
3. acceptance by the stand-alone `verify_claim`; and
4. for the two primes above the historical Montgomery-safety threshold, an additional full-interval witness that uses no Montgomery arithmetic.

The parser is fail-closed: it checks the process status, the exact p, r, k , all profile equalities, (a), (B*), and the final verdict, and rejects any output containing `DIFF`, `FAIL`, `ECHEC`, or a negative verdict. Its digest binds the claim identity, profiles, criterion values, and final verdict. The manifest separately records source and binary hashes, compiler version, exact commands, and hashes of every raw output. The final Markdown and $\text{\LaTeX}$ claim tables are generated only from the exact set of thirteen accepted manifests.

This rule is stricter than the historical transcript, whose max-run checksum did not bind p and whose pasted `verify_claim` outputs were sometimes truncated. Those files remain historical evidence, not the final source of truth. The targeted PC run populated the exact thirteen-manifest set, and the fail-closed audit accepts its 67 required executions: five per claim plus the two additional Montgomery-free high-range witnesses.

Current CUDA qualification. The recorded qualification builds the committed scanner for `sm_120` and passes the complete qualification suite on the target RTX 5090. Its manifest binds the source and binary hashes, compiler, driver, GPU identity, commands, logs, exit status, and clean pre-build state. This qualifies the discovery implementation and hardware path; it does not replace any Rabung certificate check.

4.4 Campaign execution

The production interval was $[970\,000\,000, 2\,000\,000\,000)$, partitioned into 515 contiguous chunks of width 2×10^6 . Each prime was evaluated for the active colour counts $r \leq 9$ dividing $p - 1$. The

checkpoint accounts for every chunk; 514 also have matching journal records, while chunk 7 is recovered from the checkpoint alone. The surviving outputs contain 498 per-chunk histograms and one aggregate re-scan of the first 17 chunks.

A separately written CPU sieve and the scanner produce byte-identical ordered prime streams on all 515 chunks, containing 48 823 489 primes. This checks the enumeration underlying the coverage statement. It does not replace the pointwise criterion tests for a certificate. Four historical scanner fingerprints were used, with no change to the $(3, 17)$ target predicate; the complete holdout segment uses the final fingerprint. Appendix F gives the revision table, the distinction between histogram and ordered-stream checks, and the recovery of the early aggregate evidence.

The GPU method made this finite survey practicable, but the retained material does not constitute a controlled throughput study with a common repetition and telemetry protocol. We therefore report the algorithm and coverage, without using historical kernel rates as publication benchmarks. None of the numerical lower bounds depends on a performance claim.

5 Conclusion

The scale of the search and the scale of the proof are deliberately different. The GPU campaign was used to locate candidate primes across a large interval, but the mathematical object supporting each direct bound is the succinct triple (p, r, k) . Its finite Rabung hypotheses can be re-established without trusting the search engine. The search discovers candidates; the pointwise certificate, Rabung’s theorem, and the recurrence calculation support the inequalities.

The fail-closed audit accepts the final thirteen claim manifests, so the nine registered three-colour triples give direct lower bounds for $W(3, k)$ throughout $17 \leq k \leq 25$. After closing the explicitly admitted registry under length monotonicity and the Blankenship–Cummings–Taranchuk and Xu recurrences, the direct values win for $17 \leq k \leq 21$. BCT consequences win at $k = 22, 23$ and $k = 26, 27, 28$, while Berlekamp’s verified 1968 construction wins at $k = 24, 25$. The stronger Landman–Robertson textbook statement is preserved as a published comparator with unresolved proof status in our audit. It would dominate the admitted values at lengths 24–28; the five improvements at lengths 17–21 remain even when it is included among prior comparators. The earliest public evidence located attributes the four archived two-colour triples to Monroe’s Vdwnumbers.org phase-2 project and its volunteers. Their accepted manifests make our contribution a separately auditable recheck and the recurrence consequences drawn from those inputs, not their first discovery or a gap-free audit of the historical scan. The supporting evidence includes separate-source CPU rechecks.

The holdout comparison has a different status. It is an empirical check of three recorded fixed intensities against one run-valid $(3, 17)$ trajectory in one finite window. It neither proves a density law nor strengthens the direct-bound argument. Its value is narrower: it records how the three fixed forecasts behaved in one repository-recorded holdout comparison, with the disjoint increments primary and the probabilistic readings conditional on the stated model.

The computation does not assert the absence of better Rabung certificates outside the scanned interval, or global optimality of the displayed bounds. A negative literature search establishes no priority beyond the dated corpus and constructions explicitly compared here. The short certificate description, separate verification path and numerical provenance are intended to make these finite results usable and independently checkable after the search itself has ended.

6 Data, code, and reproducibility

The accompanying reproducibility package contains the manuscript sources, certificate registries, verification programs, archived execution evidence and table generators. Its development repository is [Vulkin-prog/vdw-rabung-bounds](https://github.com/Vulkin-prog/vdw-rabung-bounds). The package was separated from the earlier development repository under an explicit file allowlist. The source-import ledger preserves the

original commit identities and byte-level provenance. Reproduction instructions are provided in `REPRODUCIBILITY.md`; three distinct levels of evidence should be distinguished.

Individual triples. The registry `audit/claims.json` contains exactly thirteen succinct Rabung certificates (p, r, k) . A reader can check one triple without reproducing the campaign by rebuilding the stand-alone verifier, which re-establishes primality, the congruence and size preconditions, and conditions $(a) \wedge (B^*)$. The current exact set of thirteen fail-closed manifests binds every verdict to the source, binary, command, environment, and raw output. Its audit accepts all 67 required executions, including the two Montgomery-free high-range witnesses. This local evidence combines GPU checks with separate-source CPU rechecks; none is an independently administered replication. External rechecks remain an optional avenue for corroboration.

Derived tables and the holdout analysis. The separate registry `audit/baselines.json` records admitted published, archived, and current strict lower bounds with source locators, while `publication/bibliography-audit.json` records the search methods, version pins, priority boundaries, and the published statement excluded from the admitted proof registry. A separate ledger, `audit/published-unresolved.json`, retains that statement and its exact numerical specializations for priority comparisons. `tools/close_baselines.py` closes the admitted inputs under length monotonicity and the registered recurrences, preserves a provenance path for each output, and generates the tables used in Section 2. The survivor list, reconstructed holdout ledger, and generated audit reproduce the disjoint counts and model means reported in Appendices A and B, conditional on the retained event list. These artifacts reproduce the reported derivations; they do not by themselves re-evaluate every prime or establish complete campaign coverage.

The complete campaign audit. Claims about exhaustive coverage of $[9.7 \times 10^8, 2 \times 10^9)$ use the authenticated 515-chunk record. Its checkpoint covers every chunk; 514 rows also have an exact journal match, while chunk 7 is explicitly and only recovered from the checkpoint. The archive contains 498 historical per-chunk histograms for chunks 18–515. The earlier interval has no manufactured per-chunk cells: its preserved evidence is one aggregate re-scan of chunks 1–17. A separate audit compared the scanner and CPU ordered-prime streams byte for byte on all 515 chunks and found the same 48 823 489 primes. Thus the aggregate first-17 re-scan closes its stated count role, while ordered-prime identity supplies the stronger enumeration check; no complete GPU campaign rerun is required.

The archived phase-2 source is credited and located; third-party articles, code and web captures are not redistributed in this package. Private commit dates document development history, not public availability. Public priority must be dated from the public article or deposited archive.

Rebuilding and checking. The documented CPU checks validate the retained evidence and exercise small oracle cases; the paper builder performs two clean builds and requires byte-identical PDFs. These operations require no GPU. A reader who wishes to re-evaluate a large individual certificate can run the stand-alone checker on its triple; this is a separate, substantially longer calculation and is not performed by the archive checks. The publication metadata, rights choices and deposit receipt belong to the archival record accompanying the files.

Licences. Copyright 2026 Brice Pouly. The original source code is released under the MIT licence. The manuscript, its $\text{\LaTeX}$ sources and PDF, and the original data and registries are licensed under [CC BY 4.0](#). The licence texts and component map accompany the files. Third-party rights and attributions are unchanged.

A Repository-recorded holdout protocol and evidence boundary

The lower bounds and the holdout comparison have different epistemic status. The former follow from Rabung’s theorem after machine verification of its hypotheses; the latter is empirical and conditional on the count model of Appendix B. This appendix states only what the retained history can reconstruct. It does not elevate that history into an externally timestamped preregistration.

Retained freeze record. Chunks 1–180 form the training window for the (3, 17) comparison. At the cut, retained commit `2b5f9f9` records the baseline count and the three numerical intensities of Remark B.1. Its diff already contains one tracker output at chunk 181, so it does not prove that every displayed parameter preceded the first post-cut observation. A later retained commit, ordered before the chunk-250 reading in the preserved Git DAG, records the schedule 250, 300, 400, 515. No later retained revision changes the parameters. The Git objects and remotely visible DAG fix contents and relative order, but their commit dates are not an independent timestamping receipt. We therefore describe this as a *repository-recorded holdout comparison*, not a *preregistration*. The candidate forms and their parameters were selected using the training data and are not called blind.

The retained training record explains η as an empirical training-window level. It fixes $m = 8 \cdot 10^{-4}$ and $\beta = 1/2$ but does not recover an exhaustive set of candidate families, a formal objective function, or the number of exploratory trials used to choose those two values. Consequently, the appendix compares three historically fixed curves conditionally; it is not a confirmatory selection test over a prespecified model class.

At each scheduled read the live program compared the cumulative observed increment with the three displayed means, identified the closest displayed mean, and printed Wald-type discrepancies normalized by $\sqrt{N_{\text{obs}}}$ for two candidates. It was not an exact Poisson test and did not contain the deviance or dispersion analyses now reported in the paper. Two manually transcribed standardizations for the third candidate were erroneous; the counts and forecast functions were not. The full-interval correction and all later sensitivity calculations are explicitly post hoc and are collected in Appendix C; they do not retroactively become part of the recorded rule. Likewise, retaining fixed m and β prevents holdout refitting but does not remove their training-window uncertainty; we make no family-wide or long-range extrapolative claim from this comparison.

One trajectory, four scheduled views. The four cumulative readings share their early data. Table 6 therefore presents the four disjoint increments as its primary view and retains the cumulative values only as an audit trail. The model-conditional interpretation of those increments is stated with the result in Section B.3.

Source and predicate checks. The 515 campaign chunks span eight git revisions and four scanner-source fingerprints. The final fingerprint starts at chunk 79, before the training cut, so the complete post-freeze comparison uses one scanner source. This removes scanner-source changes as a confounder for the recorded holdout comparison; descriptive whole-campaign summaries must nevertheless be accompanied by the full revision table and by tests across the earlier transitions.

The empirical reconstruction uses the retained event list and the campaign artifacts described below. It does not treat a historical sampled-survivor cross-check as an additional quantified validation result when its complete sample list and raw output are unavailable. Checking reported survivors would in any case address false positives, not omissions, enumeration, dispersion or temporal dependence.

Reproducibility boundary. The versioned 1859-entry list of reported run-valid (3, 17) primes, labelled complete by the historical analysis, is sufficient to reconstruct all 515 success counts and the disjoint increments in Table 6, conditional on that list. It does not by itself re-evaluate the max-run predicate or prove that no run-valid prime was omitted. The historical live tracking

script also read a campaign checkpoint that was not part of ordinary source tracking. The recovered publication package now preserves an immutable checkpoint and campaign journal, the 498 retained per-chunk histograms for chunks 18–515, the audited aggregate re-scan covering chunks 1–17, and the machine-readable revision map. The aggregate re-scan restores histogram coverage over the first 17 chunks, but not a fictitious 17-way decomposition; unavailable historical per-chunk streams are declared unavailable rather than reconstructed. The git history alone remains sufficient only for the narrow (3, 17) event-count analysis; the archived evidence is required for campaign-level diagnostics.

This boundary preserves the reconstructed record as executed, makes later repairs visible as repairs, and limits the conclusions to what the released evidence can support.

B Holdout check for run-valid (3, 17) primes

The campaign preserved the run-valid survivor event list and aggregate counts, but not the complete per-prime max-run profile. We use those data only to compare three recorded forecasts beyond a fixed training cut. The comparison is not a theorem, a universal density law, or evidence that any Rabung search has been exhausted; every probabilistic statement is conditional on the stated count model.

B.1 Population and mirror reference

A prime is *run-valid* for (r, k) when condition (a) holds, that is, $\text{maxrun}(p, r) < k$, and *valid* when $(a) \wedge (b)$ holds. The estimand here is the run-valid population. The bounds of Section 2 use the full criterion. Because condition (b) is not represented in the versioned event list, it cannot be absorbed into the density correction.

The symmetry

$$c(p - x) = c(x) + c(-1) \pmod{r}$$

leaves only the first $(p - 1)/2$ positions free. The natural i.i.d. reference for the maximal run therefore has length $n = (p - 1)/2$, not $p - 1$. This descriptively accounts for most of the reference-length shift, but the survivor list does not support a numerical mean-run claim. For $r = 3$ the case $c(-1) = r/2$ cannot occur because r is odd; more generally that case exists only for even r .

B.2 Frozen intensity models

The elementary Poisson-clumping reference is

$$\Lambda_{r,k}(p) = \frac{(r-1)(p-1)}{2r^k}, \quad g_0(p) = e^{-\Lambda_{r,k}(p)}.$$

For an isolated prime this changes on the scale r^k . The additional factor k comes only from the extreme over the available primes. Indeed, with $a_r = (r - 1)/2$ and fixed r , the heuristic number of survivors beyond x is

$$M(x) \approx \int_x^\infty \frac{e^{-a_r t/r^k}}{\varphi(r) \log t} dt \sim \frac{r^k}{a_r \varphi(r) \log x} e^{-a_r x/r^k}.$$

Balancing $M(x) \asymp 1$ gives $a_r x/r^k = k \log r - \log k + O_r(1)$, hence the qualitative extreme-value scale $x \asymp k r^k$. We do not use the local fit to predict precise numbers of future primes, declare a well exhausted, or assign narrow uncertainty intervals far outside the fitted window.

The retained program instead used

$$\tilde{\Lambda}_{r,k}(p) = \frac{(r-1)p}{2r^k}, \quad \tilde{g}_0(p) = e^{-\tilde{\Lambda}_{r,k}(p)},$$

so $\tilde{\Lambda} - \Lambda = (r - 1)/(2r^k)$. We retain this convention to reproduce the recorded calculation while keeping Λ as the natural half-length reference.

Remark B.1 (Domain of the empirical comparison). The two corrected candidates are

$$\tilde{g}_1(p) = e^{-\tilde{\Lambda}_{3,17}(p)} + m e^{-\beta \tilde{\Lambda}_{3,17}(p)}, \quad \tilde{g}_2(p) = e^{-\tilde{\Lambda}_{3,17}(p)} + \eta,$$

with $m = 8 \cdot 10^{-4}$, $\beta = 1/2$, and $\eta = 1.62 \cdot 10^{-5}$. These are local intensity models for condition (a) in the observed deep-tail window. In particular, $\tilde{g}_1(p) \rightarrow 1 + m > 1$ as $\tilde{\Lambda} \rightarrow 0$, so it is not a global probability law. The parameters were selected on the training window; freezing them prevents holdout tuning but does not remove their estimation uncertainty.

B.3 Repository-recorded holdout comparison

After chunk 180, when 1727 run-valid $(3, 17)$ primes had been observed, the retained history records the three candidates $\tilde{g}_0, \tilde{g}_1, \tilde{g}_2$ under the evidence boundary of Appendix A.

For

$$I_j = [970\,000\,000 + (j - 1) 2\,000\,000, 970\,000\,000 + j 2\,000\,000)$$

and a set A of chunks, the corrected mean derived from a recorded intensity is

$$\mu_\ell(A) = \sum_{j \in A} \int_{I_j} \frac{\tilde{g}_\ell(p)}{2 \log p} dp, \quad \ell \in \{0, 1, 2\}.$$

The factor $1/(2 \log p)$ is the analytic density used for primes $p \equiv 1 \pmod{3}$. Table 6 reports the repository-recorded scheduled readings as disjoint increments.

Table 6: Repository-recorded scheduled $(3, 17)$ readings, reported as disjoint increments. The model columns are means integrated over the full half-open chunk intervals.

chunks	observed	bare $\tilde{g}_0$	decaying $\tilde{g}_1$	constant floor $\tilde{g}_2$
181–250	90	68.35	80.26	122.20
251–300	20	18.75	24.04	57.06
301–400	16	12.58	18.59	88.87
401–515	6	2.81	5.82	90.04
total	132	102.49	128.72	358.17

The corresponding cumulative observations are 90, 110, 126, 132. They are four views of one trajectory, not four replications. The fixed decaying forecast is close to the observed total, the recorded constant-floor candidate substantially overpredicts it, and the bare reference underpredicts it. The increments are non-overlapping but are conditionally independent only under the specified inhomogeneous-Poisson model; disjointness alone is not evidence of independence, and a conditional Poisson tail is not a model-robust significance level. Appendix C records the endpoint correction and retrospective sensitivity checks.

B.4 Summary

The mirror symmetry supplies a useful first-order reference, and one fixed decaying correction tracked the held-out $(3, 17)$ trajectory more closely than the bare and constant-floor candidates. This is an empirical forecast check for one target and one finite window, not four confirmation datasets, a proof of a universal two-exponential law, or a quantitative certificate that any future Rabung search is empty.

C Retrospective diagnostics for the holdout comparison

This appendix records corrections and sensitivity analyses performed after the repository-recorded scheduled reads. They test the recorded comparison on its stated finite window; they do not alter the historical rule or enlarge the scope of the empirical claim.

Remark C.1 (Recorded readout and post hoc diagnostics). Three layers must be kept distinct. First, the recorded inputs were the three intensity functions, their numerical parameters, the baseline count, and—before the first scheduled read—the reading times. Second, the live tool selected the closest displayed mean and printed Wald-type discrepancies $(N - \mu)/\sqrt{N}$ for the bare and constant-floor candidates. It did not compute deviance residuals or exact Poisson tails; two manually noted standardizations for the decaying candidate were later found to use the wrong denominator.

Third, all deviance, dispersion, autocorrelation, negative-binomial, and block calculations are retrospective. The archival script used chunk midpoints as the two endpoints of a cumulative integral. Table 6 corrects that convention by integrating the full observed half-open chunk intervals. This changes no observation and no recorded operational intensity function, only the integration limits. The corrected means are therefore post-campaign recalculations of the recorded functions. Replacing the operational $\tilde{\Lambda} = p/3^{17}$ by the natural $\Lambda = (p - 1)/3^{17}$ changes every holdout total by less than $9 \cdot 10^{-7}$ and changes no cell of Table 6 at two decimal places.

Dispersion and dependence. On the 335 post-cut chunks, the recorded decaying model gives Pearson statistic $X^2 = 324.2$ (quasi-Poisson scale $X^2/335 = 0.968$); an NB2 sensitivity fit puts the additional-dispersion parameter at the Poisson boundary, and the Poisson deviance is 210.46. The Pearson residuals have Ljung–Box $Q_{12} = 7.17$ (asymptotic conditional reference $p = 0.85$), and aggregation into blocks of 5, 10, 20 chunks gives Pearson scales 0.81, 0.96, 1.05. These checks reveal no gross overdispersion or serial correlation for that fixed model, but they remain post hoc and have limited power in a sparse, non-stationary tail.

Block sensitivity of the forecast ordering. A moving-block resampling of the per-chunk log-score difference (circular blocks of length 10, 20 000 seeded replicates) gives a mean decaying-minus-bare advantage of 0.01185 per chunk, with a 95% percentile interval $[-0.00509, 0.02957]$ and bootstrap fraction 0.092 at or below zero. For decaying minus the recorded constant-floor model, the corresponding interval is $[0.31298, 0.49363]$ and no replicate is non-positive. Thus the former comparison is less decisive than the latter. We report the forecast ordering, not a universal fitted family or a discovery-level p -value.

D Certificate evidence identities

The following table identifies the archived executions supporting Theorem 2.1. Full commands, raw streams and environment records are preserved in the reproducibility package. The repeated GPU and CPU checks are internal qualification runs; they are not independent external replications. The identities are generated from the accepted manifests.

Table 7: Canonical fail-closed audit view of the thirteen accepted direct claims. The GPU column records the two captured structured checks; the separate-source column records the separate CPU criterion, stand-alone verifier, and Montgomery-free witness when required. Each claim is followed by its full claim, profile, and manifest identities. The JSON and Markdown views retain the full commands, artifacts, and provenance.

Claim and certificate	Provenance	GPU triplet	Separate-source checks
$W(2, 25) > 27\,333\,622\,969$ $p = 1\,138\,900\,957$	Monroe phase-2 archive; rediscovery and verification here	V2a, B7, and CPU walk PASS : 2 captures $\times$ 2 internal repeats	criterion: PASS (2) stand-alone verifier: PASS (1) Montgomery-free: <i>not required</i>
<i>Evidence identity</i> : claim bf88b3561b1055ed profile 10d669712e712d41 manifest SHA-256 7275796530ba538fedc28f605c2f0c5446029bc9322c589e70232d4341915ab6			
$W(2, 26) > 53\,126\,634\,776$ $p = 2\,125\,065\,391$	Monroe phase-2 archive; verification here	V2a, B7, and CPU walk PASS : 2 captures $\times$ 2 internal repeats	criterion: PASS (2) stand-alone verifier: PASS (1) Montgomery-free: <i>not required</i>
<i>Evidence identity</i> : claim 1c7f08f0acab68c9 profile c3a75fee4dcaec1d manifest SHA-256 e832997a47b714f630713462f81691c3d047dca8f8d3917fb4b1cf1ac9ea5b72			
$W(2, 27) > 89\,955\,478\,679$ $p = 3\,459\,826\,103$	Monroe phase-2 archive; verification here	V2a, B7, and CPU walk PASS : 2 captures $\times$ 2 internal repeats	criterion: PASS (2) stand-alone verifier: PASS (1) Montgomery-free: PASS (1)
<i>Evidence identity</i> : claim 8d4d7ce43ba93beb profile fd1f1f42dd3743e8 manifest SHA-256 be406ad31fdcedee0d692dcb1547d05e932d52b1af9f7aeacdd43e7713d8662			
$W(2, 28) > 93\,871\,785\,142$ $p = 3\,476\,732\,783$	Monroe phase-2 archive; verification here	V2a, B7, and CPU walk PASS : 2 captures $\times$ 2 internal repeats	criterion: PASS (2) stand-alone verifier: PASS (1) Montgomery-free: PASS (1)
<i>Evidence identity</i> : claim a91a6ffd87cd51fb profile 0d3a489c3f695093 manifest SHA-256 e31f02c689c6112c0b6dcf6bb89e0a4aaa24c4d5557f03d19edcf4bceea94b25			
$W(3, 17) > 31\,385\,622\,833$ $p = 1\,961\,601\,427$	this scan	V2a, B7, and CPU walk PASS : 2 captures $\times$ 2 internal repeats	criterion: PASS (2) stand-alone verifier: PASS (1) Montgomery-free: <i>not required</i>
<i>Evidence identity</i> : claim e38c156055ebf346 profile 339c507f3866bd00 manifest SHA-256 6186996a32e744488b1ef4e851f93ffded0b12a1e5ddf9c318ffcfef84c5b0a59			
$W(3, 18) > 33\,999\,912\,468$ $p = 1\,999\,994\,851$	this scan	V2a, B7, and CPU walk PASS : 2 captures $\times$ 2 internal repeats	criterion: PASS (2) stand-alone verifier: PASS (1) Montgomery-free: <i>not required</i>
<i>Evidence identity</i> : claim fba1eb1c37965218 profile 2c77bd09c2ef399f manifest SHA-256 a4750b78b870e1688a8310df808f92155a2d8ea3fd030f6b2cd33ed88d431d3b			
$W(3, 19) > 35\,999\,998\,687$ $p = 1\,999\,999\,927$	this scan	V2a, B7, and CPU walk PASS : 2 captures $\times$ 2 internal repeats	criterion: PASS (2) stand-alone verifier: PASS (1) Montgomery-free: <i>not required</i>
<i>Evidence identity</i> : claim e47f1bd4b7e28216 profile 03107afa6942f295 manifest SHA-256 677415b596ac06d955e39991a63a79d317be17eb008d51b9162a13fdb73d9368			
$W(3, 20) > 37\,999\,998\,614$ $p = 1\,999\,999\,927$	this scan	V2a, B7, and CPU walk PASS : 2 captures $\times$ 2 internal repeats	criterion: PASS (2) stand-alone verifier: PASS (1) Montgomery-free: <i>not required</i>
<i>Evidence identity</i> : claim 15f9a541dbf03f56 profile 03107afa6942f295 manifest SHA-256 4aabe71e626081502e7a2855b88cfe6188c71846b82e534d0116810673c6efd2			
$W(3, 21) > 39\,999\,998\,541$ $p = 1\,999\,999\,927$	this scan	V2a, B7, and CPU walk PASS : 2 captures $\times$ 2 internal repeats	criterion: PASS (2) stand-alone verifier: PASS (1) Montgomery-free: <i>not required</i>
<i>Evidence identity</i> : claim 68e89a17cf6a40a3 profile 03107afa6942f295 manifest SHA-256 4d41120c0394b31b2363383bbd737519df69db3bdaf45f795bfc662fcf8dedf			

Continued on next page

Table 7 continued

Claim and certificate	Provenance	GPU triplet	Separate-source checks
$W(3, 22) > 41\,999\,998\,468$ $p = 1\,999\,999\,927$	this scan	V2a, B7, and CPU walk PASS : 2 captures $\times$ 2 internal repeats	criterion: PASS (2) stand-alone verifier: PASS (1) Montgomery-free: <i>not required</i>
<i>Evidence identity</i> : claim 779a5c1362c9097e profile 03107afa6942f295 manifest SHA-256 a934f48d5930a635e7990c9358cf83659dc0c0c57eaac2bbff991fa2db900f04			
$W(3, 23) > 43\,999\,998\,395$ $p = 1\,999\,999\,927$	this scan	V2a, B7, and CPU walk PASS : 2 captures $\times$ 2 internal repeats	criterion: PASS (2) stand-alone verifier: PASS (1) Montgomery-free: <i>not required</i>
<i>Evidence identity</i> : claim 62bfff53327b6bfe profile 03107afa6942f295 manifest SHA-256 2689f5a9094e8415b28f1d11f8deca1fc9a6a217cc18af530e7091bfdae68941			
$W(3, 24) > 45\,999\,998\,322$ $p = 1\,999\,999\,927$	this scan	V2a, B7, and CPU walk PASS : 2 captures $\times$ 2 internal repeats	criterion: PASS (2) stand-alone verifier: PASS (1) Montgomery-free: <i>not required</i>
<i>Evidence identity</i> : claim ee9f28861146d0c3 profile 03107afa6942f295 manifest SHA-256 6eba765402072a653b76082db3a627641983ef48e36d370039ae580822101d3f			
$W(3, 25) > 47\,999\,998\,249$ $p = 1\,999\,999\,927$	this scan	V2a, B7, and CPU walk PASS : 2 captures $\times$ 2 internal repeats	criterion: PASS (2) stand-alone verifier: PASS (1) Montgomery-free: <i>not required</i>
<i>Evidence identity</i> : claim 54d41462f53ebdc5 profile 03107afa6942f295 manifest SHA-256 4e405a8d1bd896458d6f93bc833a40a472b2a1aa8a18e485eedc9f4249ea41c1			

E Source catalogue and comparison scope

This appendix records the source choices underlying Section 2. The catalogue fixes the scope of the finite comparison; it is not a claim that every published lower bound has been recovered.

The admitted numerical seed set was assembled on 29 August 2026. The source review was updated on 26 September 2026 without changing those seeds. It consists of the seeds listed in `audit/baselines.json`: all seventeen finite three-colour specializations of Berlekamp’s Theorem 1 from length 12 through 28, Rabung–Lotts’ direct and prime-ring length-12 row, Liang et al.’s ordinary rows from length 12 through 23 and Xu-admissible prime ring rows from lengths 13 through 23, Monroe’s version-pinned pre-window three-colour and later two-colour rows, the immutable direct tally distributed with his code, the twelve version-pinned explicit finite products of Campos, Fox and Schildkraut, the twelve explicit three-colour specializations of Gasarch and Haeupler, four archived phase-2 rows, and the thirteen claims of this work. The exact rows at lengths 17 through 20 that Monroe’s summary credits to Liang et al. are sourced to that earlier 2012 paper [12], not reassigned to the later table. Monroe’s rows at lengths 21 through 23 are pinned to arXiv v1 of 9 March 2016, while the rows at lengths 24 and 25 first appear in v4 of 6 November 2017 [13, 14]. Published entries require a stable primary locator. An unpublished or archived entry is included only with an immutable source identity and dated attribution; a merely announced value is not silently promoted to a seed. The generated output embeds this catalogue, SHA-256 hashes of both registries, and the winning provenance paths. Accordingly, “strongest” below always means strongest verified consequence of these explicitly admitted inputs, not global optimality over an unspecified literature corpus.

Literature and priority audit. Beginning the closure at length 12 is deliberate. Rabung and Lotts’ Table 1 gives prime 7 194 013 and $W(3, 12) > 79\,134\,144$ [20]. Monroe’s arXiv v1 is the first versioned public source located for the registered direct rows at lengths 13 through 15 and a then-best length-16 row; v4 first replaces the latter by prime 612 400 081 and $W(3, 16) > 9\,186\,001\,216$ [13, 14]. The registry retains the superseded v1 length-16 row as well. Every

corresponding prime and strict-bound identity is recomputed; their strongest monotone consequence at length 17 remains below the direct certificate of this work. Thus the target window is not insulated from an unexamined earlier winner, while the later journal table is used as a stable record rather than as the source of priority.

F Campaign coverage audit

The authenticated coverage-evidence package shows that the production run was configured to scan every prime in $[9.70 \times 10^8, 2.0 \times 10^9]$ (each prime tested for every $r \leq 9$ dividing $p - 1$), starting just above the largest published prime 969 347 371 of Monroe’s distributed scan [18], in a single multi- r pass ($r \leq 9$). (The narrow interval $[969\,347\,371, 9.70 \times 10^8]$ was therefore not scanned; it is immaterial, since any valid prime in it would give a bound below every one we report.) The historical checkpoint reports that the range was processed in 515 restartable chunks of width 2×10^6 , each with a fixed walltime cap. The registered revision map spans *eight* git revisions and *four* scanner-source fingerprints. Their exact allocation and the scientific effect of each transition are generated from the audit registry below. The (3, 17) max-run statistic and

source hash	chunks	revisions (number of chunks)	targets	change and transition check
87f10166	1–7	ebd54ea (7)	10	Initial campaign fingerprint; fused and mirror max-run paths already active. Covered by the frozen pre-flight agreement suite.
3598aea6	8–35	657a140 (10), 185b868 (18)	10	Added the point-verification mode and record reporting; the production target predicate was unchanged. Point claims were checked by mirror, full, CPU-walk, and separate verifiers.
cca301fe	36–78	062cc3c (43)	13	Added the three two-colour bonus targets; the already-computed $r=2$ max-run profile was reused. The new bookkeeping was exercised on [1026949000, 1026950000]; the (3, 17) path was unchanged.
3b5d1049	79–515	4f9e2cf (1), c3a9b9e (274), a93cec9 (72), 7c1ba34 (90)	13	Added aggregate congruence counters; later revisions changed the driver or theory files, not scan_gpu.cu. Old/new TARGET and CHECKSUM outputs agreed exactly and an end-to-end test passed.

Table 8: Scanner-source fingerprints and Git revisions used by the 515 campaign chunks. The registry also records the active target list and Git blob identity.

its target predicate are unchanged across all four fingerprints. The first functional expansion added only (2, 26)–(2, 28), whose early online tallies are therefore incomplete; the second added congruence counters and reproduced the previous target profiles and checksums in an old/new comparison. This supports code-path comparability for the retained (3, 17) counts, although version-stratified dispersion diagnostics remain distinct from that code audit. The historical journal also records automatic cross-checks every 50 chunks: nine completed samples agreed across the redundant scan paths and separate CPU checks, while a tenth timed out and was not counted. These samples do not turn the single production campaign into a duplicated run. Checkpoint transaction mechanics are documented in `REPRODUCIBILITY.md`. The authenticated coverage-evidence package contains the immutable checkpoint, a matching journal record for 514 chunks, the checkpoint-only record for chunk 7, 498 per-chunk histograms for chunks 18–515, and the retained transition artifacts selected by the recovery ledger.

Exhaustiveness of the prime enumeration. The per-prime computation is validated by the redundancy above; the *enumeration* of primes deserves its own audit, since the density claims

of Appendix B are claims about *all* primes in the window. The scanner enumerates primes with a plain in-process sieve per chunk; chunks are contiguous by construction ($[lo + 2i \cdot 10^6, lo + 2(i+1) \cdot 10^6)$), and the checkpoint protocol resumes only at chunk boundaries, so interruptions can neither skip nor duplicate primes within the range. We verified the retained aggregate coverage counts against a separately written segmented sieve with no shared code: over the 498 chunks for which per-chunk histograms were logged (the histogram feature was added at an early restart, from $p = 1.004 \times 10^9$ on), the campaign’s per-chunk counts of primes in every congruence class $p \equiv 1 \pmod{r}$, $2 \leq r \leq 9$ —3 984 counts in all—agree with the separate-source sieve *exactly*, with zero discrepancies (47 181 875 primes on those chunks). For the first 17 chunks, which predate the histogram feature, per-class counts were not logged during the campaign; that range, $[9.70 \times 10^8, 1.004 \times 10^9)$, was therefore re-scanned after the campaign in one aggregate invocation with the post-campaign audit build (commit `c81383885bd9`; full identities are in the registry). The retained Git history places the eight separate-source counts in commit `30e8b83df3c0` before the completed re-scan transcript. This establishes relative order inside the preserved Git DAG, not an independently timestamped precommitment. A version-specific, fail-closed parser now checks the preserved transcript, whose SHA-256 is `dff4e409...de184`; it verifies every historical TARGET, candidate, COLQ, HISTO, and TOP row before comparing the eight histogram sums with those separately recorded counts.

r	$\#\{p : p \equiv 1 \pmod{r}\}$	r	$\#\{p : p \equiv 1 \pmod{r}\}$
2	1 641 614	6	821 005
3	821 005	7	273 695
4	820 559	8	410 485
5	410 333	9	273 955

Table 9: Aggregate congruence-class totals from the post-campaign re-scan of the first 17 chunks. All eight equal the separate-source counts placed earlier in the retained Git history. This relative order is not an external timestamp. It is one aggregate comparison, not 17 per-chunk comparisons and not an ordered-prime identity test.

The re-scan also lists exactly 405 run-valid (3, 17) candidates, consistently with its TARGET row and its $r = 3$ histogram. It was one invocation over the whole interval, however, so it supplies aggregate class totals rather than 17×8 per-chunk cells. These audits establish exact agreement of the counts; counts alone do not establish identity of the ordered prime lists. The audit utility therefore also emits a canonical ordered stream for byte comparison and hashing. Its final manifest records byte-for-byte scanner/CPU identity on all 515 chunks, independently of the 3 984 matching counts. The four historical scanner blobs contain a byte-identical 249-byte `sieve_primes` function (SHA-256 recorded in the registry). That function accepted an inclusive upper endpoint, whereas the audit stream is half-open; every campaign boundary is an even integer greater than two, hence composite, so the two conventions enumerate exactly the same primes on all 515 chunks. The current scanner makes the half-open convention explicit. In total the window $[9.7 \times 10^8, 2 \times 10^9)$ contains 48 823 489 primes in both byte-identical ordered streams. Across the retained 498-chunk histograms and the aggregate 17-chunk re-scan, the preserved outputs report corresponding per-class totals. The separate-source sieve counts 1 448 736 primes in $[9.7 \times 10^8, 10^9)$, hence 47 374 753 in $[10^9, 2 \times 10^9)$. This equals the externally published difference $\pi(2 \times 10^9) - \pi(10^9) = 98\,222\,287 - 50\,847\,534$ from Oliveira e Silva’s table [19]; it checks the interval difference, not either absolute value separately.

The authenticated checkpoint and journal manifest accounts for all 515 contiguous chunks without inventing the absent chunk 7 journal row; the saved driver ordering establishes why that row can be recovered from the checkpoint. Together with the 515-chunk ordered-prime identity audit, this closes the local coverage audit while preserving the distinction between the 498 per-chunk histograms and the one aggregate re-scan of chunks 1–17. The strongest candidate

direct bounds located by the scan were passed to Section 4.3; no bound was recorded from the scanning pass alone.

Acknowledgements

This work rests on Daniel Monroe’s distributed computation [18] and on the code and direct tally released with it [15], which served as the baseline against which our engine was validated. We thank Daniel Monroe and the volunteer contributors to the Vdwnumbers.org BOINC project for the phase-2 computation whose archived table supplied the four historical two-colour candidates [16] (Section 2). We also thank the Internet Archive for preserving dated public access to the surviving project records.

Declaration on artificial-intelligence assistance

The research was organized by the author with substantial assistance from artificial-intelligence systems—Anthropic’s Claude (models `claude-fable-5` and `claude-opus-4-8`) and OpenAI Codex in ChatGPT Work, including GPT-6 Astra for the final revision—used to explore and design the computational method, plan and analyse the campaign, draft and revise the manuscript, examine the literature and provenance, and carry out separate counter-audits. Under the author’s direction, these systems created or substantially developed all original computational code for the project: the GPU scanner, campaign drivers, reference implementations and CPU verifiers, statistical- and audit-analysis programs, recovery and operator tooling, tests, and reproducibility and release tooling.

The sole exception among computation code used or consulted during the project is pre-existing code recovered from Daniel Monroe’s project. It is treated as an attributed upstream source, its provenance is tracked, and it is not redistributed in this repository; no claim of authorship is made for it.

These procedures constitute neither an independent scientific validation nor a human certification of the computations. No independently administered external replication has been performed. The local rechecks use separately written source code and distinct arithmetic paths but remain author-controlled and are not independent replications. The author conceived the project, set its scope and claims, provided and operated the hardware, and alone assumes responsibility for the manuscript, the code and data attributed to this project, the references, the conclusions, and their possible errors and limitations.

References

- [1] Elwyn R. Berlekamp. A construction for partitions which avoid long arithmetic progressions. *Canadian Mathematical Bulletin*, 11(3):409–414, 1968. doi: 10.4153/CMB-1968-047-7.
- [2] Thomas Blankenship, Jay Cummings, and Vladislav Taranchuk. A new lower bound for van der Waerden numbers. *European Journal of Combinatorics*, 69:163–168, 2018. doi: 10.1016/j.ejc.2017.10.007.
- [3] Marcelo Campos, Jacob Fox, and Carl Schildkraut. A new lower bound for two-color van der Waerden numbers. arXiv:2608.20824v1, August 2026. URL <https://arxiv.org/abs/2608.20824v1>. Version 1, 21 August 2026.
- [4] Jacob Fox and Zach Hunter. Three-color van der Waerden numbers grow super-exponentially. arXiv:2606.02541v1, June 2026. URL <https://arxiv.org/abs/2606.02541v1>. Version 1, 1 June 2026.
- [5] William Gasarch and Bernhard Haeupler. Lower bounds on van der Waerden numbers: randomized- and deterministic-constructive. *The Electronic Journal of Combinatorics*, 18(1):P64, 2011. doi: 10.37236/551.

- [6] P. R. Herwig, M. J. H. Heule, P. M. van Lambalgen, and H. van Maaren. A new method to construct lower bounds for van der Waerden numbers. *The Electronic Journal of Combinatorics*, 14(1):R6, 2007. doi: 10.37236/925.
- [7] Zach Hunter. Lower bounds for multicolor van der Waerden numbers. *Israel Journal of Mathematics*, 267:783–795, 2025. doi: 10.1007/s11856-025-2735-0. Published online 17 February 2025.
- [8] Karen R. Johannson. Variations on a theorem by van der Waerden. Master’s thesis, University of Manitoba, Winnipeg, Manitoba, May 2007. URL <https://hdl.handle.net/1993/321>. Master of Science thesis.
- [9] Jakub Kozik and Dmitry Shabanov. Improved algorithms for colorings of simple hypergraphs and applications. arXiv:1409.6921v1, 2014. URL <https://arxiv.org/abs/1409.6921v1>. Theorem 2; version 1, 24 September 2014.
- [10] Bruce M. Landman and Aaron Robertson. *Ramsey Theory on the Integers*, volume 24 of *Student Mathematical Library*. American Mathematical Society, Providence, RI, 2004. doi: 10.1090/stml/024.
- [11] Bruce M. Landman and Aaron Robertson. *Ramsey Theory on the Integers*, volume 73 of *Student Mathematical Library*. American Mathematical Society, Providence, RI, 2 edition, 2014. doi: 10.1090/stml/073.
- [12] Meilian Liang, Xiaodong Xu, Zehui Shao, and Baoxin Xiu. Lower bounds on some van der Waerden numbers based on quadratic residues. *Journal of Combinatorial Mathematics and Combinatorial Computing*, 83:23–31, 2012. URL <https://combinatorialpress.com/jmcc-articles/volume-083/lower-bounds-on-some-van-der-waerden-numbers-based-on-quadratic-residues/>.
- [13] Daniel Monroe. New lower bounds for van der Waerden numbers using distributed computing. arXiv:1603.03301v1, March 2016. URL <https://arxiv.org/abs/1603.03301v1>. Version 1, 9 March 2016.
- [14] Daniel Monroe. New lower bounds for van der Waerden numbers using distributed computing. arXiv:1603.03301v4, November 2017. URL <https://arxiv.org/abs/1603.03301v4>. Version 4, 6 November 2017.
- [15] Daniel Monroe. Direct Rabung tally distributed with the van der Waerden search code. Versioned project data file, 2019. URL <https://github.com/hmonroe/vdw/blob/7000ad7d99cb779ae82b495d709f4a4e23fa7eef/output.txt>. Immutable file revision introduced 6 March 2019; detailed provenance in the accompanying registry.
- [16] Daniel Monroe. BOINC phase-2 bounds table. Internet Archive capture of <http://vdwnumbers.org/bounds.txt>, 2021. URL https://web.archive.org/web/20220428101631id_/http://vdwnumbers.org/bounds.txt. Source header dated 18 August 2021; surviving final capture of 28 April 2022; detailed chronology and hashes in the accompanying registry.
- [17] Daniel Monroe. Projects. Personal website, <https://daniel-monroe.github.io/projects.html>, 2025. URL <https://github.com/daniel-monroe/daniel-monroe.github.io/blob/825778cca5e669cbf60140e35170113603010076/projects.html>. Immutable revision of 28 March 2026; accessed 29 August 2026.
- [18] Daniel Monroe. New lower bounds for Van der Waerden numbers using distributed computing. *Journal of Combinatorial Mathematics and Combinatorial Computing*, 128:305–315, 2026. doi: 10.61091/jmcc128-19. Published online 4 December 2025.
- [19] Tomás Oliveira e Silva. Tables of values of $\pi(x)$ and of $\pi_2(x)$. University of Aveiro; data file <https://sweet.ua.pt/tos/primes/1d09.txt.gz>, 2005. URL https://web.archive.org/web/20160704031247id_/http://sweet.ua.pt/tos/primes/1d09.txt.gz. Table of $\pi(k \cdot 10^9)$ last updated 23 January 2005; accessed 28 August 2026.
- [20] John Rabung and Mark Lotts. Improving the use of cyclic zippers in finding lower bounds for van der Waerden numbers. *The Electronic Journal of Combinatorics*, 19(2):P35, 2012. doi: 10.37236/2363.
- [21] John R. Rabung. Some progression-free partitions constructed using Folkman’s method. *Canadian Mathematical Bulletin*, 22(1):87–91, 1979. doi: 10.4153/CMB-1979-012-1.
- [22] Jonathan Sorenson and Jonathan Webster. Strong pseudoprimes to twelve prime bases. *Mathematics of Computation*, 86(304):985–1003, 2017. doi: 10.1090/mcom/3134.
- [23] Xiaodong Xu. A generalization of sets without long arithmetic progressions based on Szekeres algorithm. *Journal of Number Theory*, 133(11):3670–3677, 2013. doi: 10.1016/j.jnt.2013.05.008.